

LakeTemperature Testing Software

Slides by Aidan Bensch

Background

- We are testing a module of *E3SM*,¹ an environmental model. This module, *LakeTemperature*, is part of the greater *ELM* (*E3SM* Land Model) which was originally a fork of the *CLM* (Community Land Model).
- We are able to test specifically this module using *SPEL*,² a software that adds a *NetCDF*³ interface for the constants, inputs, and outputs of a module of *E3SM* and compiles that specific part of the model. The executable produced by this is named `elmtest.exe`.

1. "E3SM - Energy Exascale Earth System Model," E3SM - Energy Exascale Earth System Model, Nov. 18, 2025. <https://e3sm.org/> (accessed Feb. 03, 2026).

2. peterdschwartz, "GitHub - peterdschwartz/SPEL_OpenACC: Python tool designed for E3SM Land Model to create unit-tests and code-insertion of GPU compiler directives.," GitHub, 2025. https://github.com/peterdschwartz/SPEL_OpenACC (accessed Feb. 03, 2026).

3. "Unidata | NetCDF," Ucar.edu, 2023. <https://www.unidata.ucar.edu/software/netcdf>

Project Goals

- Develop a software that can systematically change the constants used in the LakeTemperature model and allow us to visualize and observe patterns in how this affects its output.
- Apply testing techniques to try and find values for constants that will cause physically impossible or otherwise erroneous outputs. Apply assertions pertaining to the values of constants, inputs, and outputs based around physical laws and other constraints provided by domain scientists.

Our Testing Solution

- Automates the process of modifying constants in spel-constants0001.nc and the process of analyzing the resulting values in fut-outputs0001.nc
- Both the process of choosing values for each constant and the process of analyzing the resulting outputs is extendable through sampling plugins and analysis plugins respectively.

Constants, Inputs, and Outputs

- Constants control how the model behaves and manipulates data.
- Inputs provide starting data for the model.
- Outputs include data that the model has generated.
- A variable can be both an input and an output in some instances.
- We are working with 25 out of the 28 constants, 26 inputs, 9 outputs, and 8 that are both inputs and outputs.

NetCDF

- Network Common Data Form (*NetCDF*) is a format for self-describing datasets. Each file is composed of “dimensions” and “variables.” Variables are multi-dimensional arrays, with the meaning of each dimension described in the file.

```
netcdf x_times_y {  
  dimensions:  
    x = 5 ;  
    y = 5 ;  
  variables:  
    float x(x) ;  
    float y(y) ;  
    float z(x, y) ;  
  data:  
  
    x = -2, -1, 0, 1, 2 ;  
  
    y = -4, -2, 0, 2, 4 ;  
  
    z =  
      8, 4, -0, -4, -8,  
      4, 2, -0, -2, -4,  
      -0, -0, 0, 0, 0,  
      -4, -2, 0, 2, 4,  
      -8, -4, 0, 4, 8 ;  
}
```

Sampling Plugins

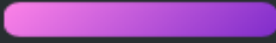
SampleGroup(s)

Testing

Tables of Testing Data

Analysis Plugins

Key

-  = Input Data
-  = Reference Data
-  = Output Data



What is a Sample?

- A sample is a set of values for the constants contained in `spel-constants0001.nc`.
 - There are 28 constants in total. We don't yet support modification of the 3 array constants. Of the 25 scalar constants we experiment with, 8 have no observable effect on the output.
- `elmtest.exe` is run once for each sample.

What is a Sample? (Illustration)

Constants (formerly lakeparams.txt)

```
betavis
0.0
za_lake
0.6
n2min
7.4999999999999993E-005
tdmax
277.0
pudz
0.0
mixfact
10.0
depthcrit
25.0
```

Example Sample (JSON used for illustration)

```
{
  "betavis": 0.0,
  "za_lake": 0.6,
  "n2min": 7.4999999999999993E-005,
  "tdmax": 277.0,
  "pudz": 0.0,
  "mixfact": 10.0,
  "depthcrit": 25.0
}
```

What Outputs are Received per Sample?

- Analysis plugins will receive the outputs from the `spe1-outputs0001.nc` file in the form of multidimensional numpy arrays, which range from one to three dimensions.

Outputs Received by Analysis Plugins (JSON for illustration)

```
{  
  "veg_ef%eflx_soil_grnd": [0.0, ...],  
  "veg_ef%eflx_gnet": [[0.0, ...], ...],  
  ...  
}
```

Sample Groups

- Represents a grouping of samples, ordered—for the sake of simplicity, not necessity—to match indices with the list of model data
- Analysis plugins can analyze the result of multiple consecutive runs of `elmtest.exe` in context this way.
- Having multiple sample groups is useful because each one can cover a range of values for a specific combination of constants whose combined effect is being examined.

Sample Group Illustration

Sample Group Data (JSON for illustration)

```
[
  {
    "betavis": 0.0,
    ...
  },
  {
    "betavis": 0.25,
    ...
  },
  {
    "betavis": 0.5,
    ...
  }
]
```

Ordered Output Data (JSON for illustration)

```
[
  {
    "lakestate_vars%betaprime_col": [0.0, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.25, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.5, ...],
    ...
  }
]
```

Plugins

- Our testing framework supports two types of plugins, sampling and analysis. Sampling plugins define values for constants, and analysis plugins view the outputs of the model.

Sampling Plugin Interface

- Sampling plugins define a subclass of `Sampler`, returning one or more `SampleGroups`, which are sequences of individual samples.

Analysis Plugin Interface

- Analysis plugins can either be “per sample” or “sample group” analysis plugins.
- “Per sample” plugins define a subclass of `PerSampleAnalyzer`, whose methods are called each time `elmtest.exe` is run, and which receives the outputs produced by that specific sample.
- “Sample group” plugins define a subclass of `SampleGroupAnalyzer`, whose method is called after all samples in a group are run through `elmtest.exe`, and which receives all outputs from each.

Configuration

